

```

Logical sector size: 512 bytes
Disk identifier (GUID): 13C43157-EF81-4F47-9425-E6D6FD96399
Partition table holds up to 128 entries
First usable sector is 34, last usable sector is 976773134
Partitions will be aligned on 2048-sector boundaries
Total free space is 2014 sectors (1007.0 KiB)

```

Number	Start (sector)	End (sector)	Size	Code	Name
1	2048	616447	300.0 MiB	2700	Basic data partition
2	616448	819199	99.0 MiB	EF00	EFI system partition
3	819200	1081343	128.0 MiB	0C01	Microsoft reserved part
4	1081344	268435455	127.5 GiB	0700	Basic data partition
5	268435456	976773134	337.8 GiB	8E00	Linux LVM

In my case, the first four partitions were automatically created when I installed Windows 8 in UEFI mode. Notice that the ESP isn't the first partition on the disk. It doesn't need to be.

What you're looking for is the ESP. If you have a FAT32 partition somewhere in your system, *gdisk* should have automatically made it the ESP. If you don't have a FAT32 partition in somewhere, you need to create the ESP.

The basic idea is to have the ESP mount as */boot*. Here's what you can do:

- If you have an *ext4* */boot* partition, back up the data in it, run *mkfs.vfat -F32* on it, and copy the data back. Change the entry for */boot* in your */etc/fstab* file to read *vfat* instead of *ext4*.
- If you have no */boot* partition, create one in your unallocated space using *gdisk*, which is exactly like using *fdisk*. Start by running *sudo gdisk /dev/sdX*, then press *n* to create a new partition. Specify the starting and ending location in the next two prompts (if in doubt, just accept the defaults). When it asks to set a partition type, type in *ef00*. That's *gdisk*'s shorthand for the ESP type. Once you're done, format it with FAT32 by running *mkfs.vfat -F32* on it.

Either way, you should end up with a FAT32 boot partition. It should be 256 MB or more, although you can get away with much less.

Once you have the */boot* partition set up, copy the kernel file and the *initrd*. To be safe, make a copy of the kernel binary and attach a *.efi* extension to it. Now download a copy of the EFI shell and place it at */boot/EFI/Boot/bootx64.efi*. To do that, open a shell and run the following commands:

```

$: cd /boot
$: sudo mkdir -pv EFI/Boot
$: cd EFI/Boot
$: sudo wget -c https://github.com/tianocore/edk2/raw/master/EdkShellBinPkg/FullShell/x64/Shell_Full.efi
$: sudo mv Shell_Full.efi bootx64.efi

```

This sets the EFI shell as the default bootloader that

should boot up if nothing else is configured (which is true, in our case).

Now it's time to boot the system in EFI mode. Cross your fingers and reboot the system, then enter your system setup program (your BIOS utility).

Once you're in, look around for an option that says something like *Boot Mode* with options like *BIOS/UEFI*, *BIOS Only* and *UEFI Only*, or anything similar. Switch it to *UEFI Only*. Also, if there's an option about CSMS (Compatibility Support Modules), turn it off.

Look around for options pertaining to Secure Boot. Disable everything you see about it. We'll deal with Secure Boot later. Save your settings and reboot the system.

UEFI: First boot

Once the computer reboots, you should be dropped into a command-line shell. You need to fish around for the ESP. Start off by typing the following command:

```

> fs0:
fs0:\> dir

```

You should get a directory listing for the file system. If you can see your kernel binary and the *initrd* images that you copied, you've hit the jackpot. If not, look at the other available file systems. The EFI shell should have given you a list of file systems available to the system, at the very top of the screen. Any name starting with *fs* is a file system readable by UEFI. If you need to switch to *fs1*, you can simply type in the following code:

```

fs0:\> fs1:
fs1:\>

```

Once you find your kernel and *initrd*, you can simply boot up by typing the following:

```

fs0:\> vmlinuz-linux initrd=\initrd-linux.img root=/dev/sda2
rw quiet

```

There are a couple of things you need to note here. The first is that the path to the *initrd* image is specified using backslashes instead of forward slashes. This is because the UEFI shell uses DOS style paths, and the path to the *initrd* is passed to the UEFI Boot Services subsystem, verbatim, so that it can load it up into memory. Second, this might not always work—some UEFI implementations are very finicky and won't execute a program unless it has a *.efi* extension. This is why I asked you to keep a copy of the kernel around with the *.efi* extension.

Before you boot, add any additional parameters you pass to the kernel, and press 'Enter'. If all goes well, you should see your kernel begin to boot. Pretty soon, you should be at the desktop. Congratulations! You are now using UEFI!